Java Order Service + AI Agent + MCP

Implementation requirements - reviewed and expanded (v2)
Scope: minimum viable read flow, followed by safe write flow. This document refines the supplied three-page architecture; example endpoints and schemas are proposed contracts, not existing APIs.

1. Purpose and non-negotiable boundaries

Expose selected Order Service capabilities to an AI Agent via MCP without relocating business rules. The Agent interprets user intent and orchestrates tools; MCP handles protocol and contract mapping; Java remains authoritative for authentication/authorization decisions, business validation, transactions, persistence, concurrency and error semantics.

Reference path: User -> AI Agent (LLM + MCP Client) -> MCP Server/Adapter -> Java Order API / Application Service -> Domain -> Repository / Database. Responses travel back along the same path.

2. Deployment and protocol requirements

• Support two deployment models: (A) separate MCP Server calling an authenticated Java API; (B) embedded MCP Server calling Java application services. Select one before implementation. The reference MVP assumes A.

• For remote MCP, use Streamable HTTP and standard MCP initialization, tool discovery (tools/list) and invocation (tools/call). For local development, stdio is an alternative; do not confuse MCP transport with MCP-to-Java REST.

• MCP Client is hosted by the Agent application. Agent needs an LLM supporting tool use, an MCP connection configuration, and an execution loop that handles tool results and errors.

• Define service URLs, timeouts, credentials, environment configuration, supported protocol version and deployment ownership.

3. Minimal functional scope

Stage	Required behavior	Out of scope
Read-only MVP	search_orders: fetch recent accessible orders; display result and empty/error states	RAG, vector DB, Kafka, workflow engine
Write extension	create_order with confirmation, authorization, domain validation, idempotency and status recovery	Unrestricted CRUD or direct database access

4. Read flow: show my last five orders

• 1. User requests recent orders in natural language.

• 2. Agent initializes MCP connection, discovers search_orders, and selects it.

• 3. Agent invokes search_orders with {limit: 5, sort: "createdAt,DESC"}; user/tenant identity is NOT taken from LLM arguments.

• 4. MCP authenticates the caller, validates tool schema, propagates trusted context and correlation ID, then calls the Java API.

• 5. Java independently authenticates/authorizes, constrains results to the caller’s account/tenant scope, invokes OrderService and queries repository.

• 6. Java returns typed order data or a stable error; MCP maps it to a tool result; Agent displays orders without inventing missing details.

5. Write flow: create a limit BUY order

• 1. User requests: BUY 10 AAPL at limit 200 USD for an authorized account.

• 2. Agent collects missing required fields and displays the exact proposed action; obtain explicit user confirmation before tool invocation.

• 3. Agent invokes create_order with typed fields and a stable idempotency key for that user action. Confirmation is orchestration, not authorization.

• 4. MCP validates syntax, authenticates the caller and forwards trusted identity, scope, request ID and idempotency key.

• 5. Java checks permission and target account access, validates current business conditions, order type, quantity and price; it must not trust prior reads or the Agent's judgment.

• 6. Java executes the operation within its own transaction, using existing concurrency controls and persistent idempotency handling.

• 7. Java returns orderId, status and operationId (or a typed failure); MCP passes it to Agent; Agent reports only the verified outcome.

• 8. On timeout or unknown outcome, Agent must check operation/order status before retrying with the same key; never claim failure or place a second order blindly.

6. Tool contracts - minimum proposed fields

Tool	Input	Output
search_orders	limit (1..100), optional status/sort; optional accountId only when backend verifies access	orders[], count, optional nextCursor
create_order	accountId, symbol, side, quantity, orderType, limitPrice when LIMIT, idempotencyKey	orderId, status, operationId, createdAt
get_order / get_operation_status	orderId or operationId; authorized scope	current status; useful for unknown outcomes

Input/output contracts should use machine-readable JSON Schema, explicit required fields, bounded limits and stable semantic names. Tool descriptions must distinguish read from side-effecting actions. Treat MCP schemas as versioned integration contracts.

7. Authentication, authorization and trust boundaries

• Authenticate the MCP caller/session; authorize which MCP tools it may invoke.

• Define user-delegated authorization (preferred for user-specific orders) or a tightly scoped service identity. Never use arbitrary userId/tenantId/accountId supplied by the LLM as proof of identity.

• If MCP forwards credentials to Java, use a supported audience-scoped delegation/token exchange or a documented trusted service-to-service mechanism; do not blindly pass unrelated bearer tokens.

• Java independently enforces operation-level permission and row/account/tenant scope. Deny cross-tenant requests and unauthorized target orders.

• Protect transport with TLS where applicable; manage secrets outside prompts/logs; prevent prompt injection or tool-result text from overriding authorization and confirmation.

8. Transactions, retries and confirmation

• No transaction spans a multi-turn Agent conversation. Backend owns atomicity, concurrency, and state revalidation at execution time.

• Define idempotency key scope, payload matching, retention, duplicate responses, and concurrent duplicate behavior; reject same key with a different payload.

• Define timeouts and cancellation behavior; cancellation or network timeout does not imply rollback. Provide a status lookup for uncertain write outcomes.

• Require explicit confirmation for financial, destructive or bulk writes. Bind confirmation to the exact operation parameters; require fresh confirmation after material changes.

• Prevent duplicate submission caused by repeated Agent calls, UI clicks, MCP retries or transport reconnects.

9. Errors and response handling

Define a stable business error envelope: code, message, correlationId, optional fieldErrors and retryable flag. Categories: VALIDATION_ERROR, NOT_FOUND, ACCESS_DENIED, BUSINESS_RULE_VIOLATION, CONFLICT, EXTERNAL_SERVICE_ERROR, INTERNAL_ERROR. Separate protocol/transport errors from business failures; never leak stack traces or sensitive records to the Agent.

10. Observability and operational controls

• Propagate trace/correlation IDs across Agent, MCP and Java; log tool, actor, target, operation ID, execution channel AI_AGENT, outcome and non-sensitive parameters.

• Capture latency, success/failure, timeouts, retries, denied access and domain-rule rejections; define retention and access for audit records.

• Set rate limits, input-size limits, per-tool timeouts and maximum tool-call iterations to contain runaway Agent behavior.

• Apply least privilege and separate read/write tool availability by environment and permission.

11. Acceptance criteria and test matrix

Scenario	Expected result
Valid search, authorized user	Only scoped orders returned; Agent formats correctly
No orders / invalid arguments	Empty result / schema validation error; no hallucinated orders
Unauthorized account or tenant	Java denies access; no data exposure
Valid confirmed create	Exactly one persisted order; structured verified response
Write without confirmation	Agent does not invoke write tool
Invalid or stale order state	Backend rejects; no persistence
Duplicate call with same key	One business operation; same prior outcome

Scenario	Expected result
Same key, different payload	Conflict; no second order
Timeout after commit	Status lookup resolves outcome; no blind duplicate
MCP/API unavailable	Safe error; no false success
Prompt-injection in tool result	No permission bypass or unconfirmed write

12. Decisions required before implementation

• D1: Separate MCP Server (reference) or embedded MCP in Order application?

• D2: Which Agent runtime and LLM provider/model?

• D3: User-delegated identity, token exchange, or service identity with backend authorization mapping?

• D4: Which existing Java endpoints/service methods and order model fields are available?

• D5: Which order actions require confirmation and which environments permit writes?

• D6: How are idempotency keys and operation status persisted and queried?

13. Recommended delivery sequence

Milestone 1: Connect Agent to MCP; initialize/discover tools; call search_orders; enforce authentication and Java authorization; return typed orders/errors; pass acceptance tests for read flow.

Milestone 2: Implement create_order with confirmation, idempotency, audit, concurrency validation and timeout/status recovery; pass write acceptance tests.

14. Review conclusion

The original architectural boundaries are sound. The principal gaps were deployment/transport choice, the Agent-MCP lifecycle, concrete tool contracts, secure identity delegation, confirmation binding, timeout recovery, and testable acceptance criteria. This revision closes those specification gaps at the requirements level; D1-D6 are explicit implementation decisions rather than silently assumed facts.